

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Дата сдачи на проверку:

«__» _____ 2026 г.

Проверено:

«__» _____ 2026 г.

Разработка GUI-приложения Formula Racing Management System на языке C# с
использованием базы данных.
Отчёт по лабораторной работе №2
по дисциплине
«Объектно-ориентированное программирование»

Разработал студент гр. ИВТб-2301-05-00

_____/Черкасов А. А./
(подпись)

Преподаватель

_____/Шмакова Н. А./
(подпись)

Работа защищена

«__» _____ 2026 г.

Киров
2026

Цели лабораторной работы

- перенести иерархию классов из Лабораторной работы №1 на язык C#, сохранив все принципы ООП;
- разработать графический интерфейс пользователя (GUI) для управления объектами предметной области;
- обеспечить персистентное хранение данных в реляционной базе данных с нормализованной схемой;
- реализовать полный набор CRUD-операций и методы предметной области из первой работы.

Задание

На основании иерархии классов из Лабораторной работы №1 разработать GUI-приложение:

1. Все требования первой работы остаются в силе: каждый класс в отдельном файле, базовый класс абстрактный, наследование, доступ к полям только через свойства/геттеры-сеттеры.
2. Реализовать графический интерфейс. Платформа — Avalonia UI (кроссплатформенная альтернатива WinForms; использование согласовано с преподавателем).
3. Данные хранить в БД SQLite. Данные не скопом в одной таблице — используется стратегия «Таблица для каждого типа» (Table-Per-Type). Реализованы все основные CRUD-функции.
4. Методы классов первой работы (`CalculatePoints`, `AddPoints`, `GetWinRate`) доступны и используются в GUI.
5. В отчёте описана диаграмма классов вместе с классами форм (Views/ViewModels) а также особенности подключения к БД.

Реализация приложения

Предметная область

Система управления гоночными командами охватывает три чемпионата: Formula 1, Formula 2 и Formula E. Каждая команда может участвовать симуляции гонки с начислением очков по реальной системе, сравнением с соперниками и подсчётом побед и подиумов.

Описание иерархии классов предметной области

Иерархия классов перенесена из C++ (Лаб. 1) на C# с сохранением структуры и полиморфизма:

- `FormulaTeam` (abstract) — базовый абстрактный класс в файле `Core/FormulaTeam.cs`. Хранит общие данные: `TeamName`, `PrincipalName`, `Headquarters`, `ChampionshipPosition`, `RaceWins`, `Podiums`, `Drivers`. Все поля доступны только через C#-свойства (`get/set`). Содержит абстрактные методы `GetSeriesName()` и `CalculatePoints(int position)`, а также конкретные: `AddPoints()`, `AddWin()`, `AddPodium()`, `GetWinRate()`. Перегружены операторы `>` и `<` для сравнения команд по очкам.
- `F1Team` : `FormulaTeam` — класс Formula 1 в файле `Core/F1Team.cs`. Добавляет: `PowerUnit`, `BudgetCapMln`, `ConstructorPos`. Реализует `CalculatePoints` по системе 25 – 18 – 15 – 12 – 10 – 8 – 6 – 4 – 2 – 1. Собственные методы: `CalculateBudgetStatus()` и `GetPowerUnitStatus()`.
- `F2Team` : `FormulaTeam` — класс Formula 2 в файле `Core/F2Team.cs`. Добавляет: `ChassisModel`, `F1Graduates`, `IsFeederSeries`. Реализует `CalculatePoints` по системе 10 – 8 – 6 – ... Собственные методы: `GetFeederStatus()`, `GetGraduateStatus()`.
- `FormulaETeam` : `FormulaTeam` — класс Formula E в файле `Core/FormulaETeam.cs`. Добавляет: `EnergyPartner`, `BatteryCapacityKwh`, `SustainabilityScore`. Реализует `CalculatePoints` по системе Formula E. Собственные методы: `GetEcoRating()`, `GetBatteryEfficiency()`.
- `FormulaDriver` — вспомогательный класс в файле `Core/FormulaDriver.cs`.

Хранит данные пилота: `Id`, `TeamId`, `FirstName`, `LastName`, `RacingNumber`, `Points`. Вычисляемые свойства: `FullName`, `DisplayString`.

Описание классов графического интерфейса (Views и ViewModels)

Приложение построено по паттерну **MVVM** (Model-View-ViewModel), реализованному с помощью библиотеки `CommunityToolkit.Mvvm`. Каждое окно (View) связано со своей ViewModel через механизм привязки данных (Data Binding) Avalonia:

- `MainWindow` / `MainViewModel` — главное окно приложения. ViewModel хранит коллекцию `ObservableCollection<object> ListItems` (команды и заголовки серий), ссылку на выбранную команду `SelectedTeam`, флаги режимов отображения `IsStandingsView` и `IsDriverStandingsView`. Команды (ICommand): `DeleteTeamCommand`, `LoadTeamsCommand`. Публичный метод `DeleteTeamById(int id)` вызывается из code-behind при нажатии кнопки удаления.
- `TeamEditorWindow` / `TeamEditorViewModel` — диалоговое окно добавления и редактирования команды. ViewModel содержит поля формы, привязанные к контролам, выпадающий список серий, проверку типа команды. Метод `ToTeam()` преобразует состояние формы в объект `FormulaTeam` нужного подтипа.
- `TeamComparisonWindow` / `ComparisonViewModel` — окно сравнения двух команд. Принимает выбранную команду и полный список команд; формирует текстовое сравнение по очкам, победам и подиумам.
- `RaceSimulationWindow` — окно симуляции гонки. Позволяет ввести позицию финиша; возвращает результат `ResultPosition` вызывающему коду, который затем вызывает `CalculatePoints()` и `AddWin()`, демонстрируя использование методов из Лаб. 1.

Особенности подключения к базе данных

Для хранения данных используется **SQLite** — встроенная реляционная СУБД, не требующая установки отдельного сервера. Работа с базой данных инкапсулирована в классе `DatabaseManager` (`Database/DatabaseManager.cs`).

Подключение и инициализация

Подключение выполняется через ADO.NET-провайдер `Microsoft.Data.Sqlite`. Строка подключения задаётся в конструкторе:

<MINTED>

При каждом запуске приложения вызывается `InitializeDatabase()`, создающий таблицы (`CREATE TABLE IF NOT EXISTS`) и при необходимости — начальные данные (seeding).

Схема базы данных (Table-Per-Type)

Данные не хранятся в одной таблице. Применяется стратегия «Таблица для каждого конкретного типа»: общая таблица **Teams** плюс отдельные таблицы с дополнительными полями для каждого подтипа, связанные внешним ключом:

- **Teams**: Id, Type, Name, Principal, HQ, Points, Wins, Podiums.
- **F1Details**: TeamId (FK → Teams.Id), PowerUnit, BudgetCap, ConstructorPos.
- **F2Details**: TeamId (FK → Teams.Id), ChassisModel, Graduates, IsFeeder.
- **FEDetails**: TeamId (FK → Teams.Id), EnergyPartner, BatteryKwh, SustainScore.
- **FormulaDrivers**: Id, TeamId (FK → Teams.Id), FirstName, LastName, RacingNumber, Points.

Объектно-реляционное отображение с Dapper

Для маппинга строк результата запросов в объекты C# используется микро-ORM **Dapper**. Он не требует конфигурации сущностей, но позволяет избежать ручного разбора `SqliteDataReader`:

<MINTED>

CRUD-операции

Класс `DatabaseManager` реализует полный набор операций:

- **Create / Update** — метод `SaveTeam(FormulaTeam team)`: если `team.Id == 0` — INSERT с получением нового Id через `last_insert_rowid()`, иначе — UPDATE. Все операции обёрнуты в транзакцию.
- **Read** — метод `GetAllTeams()` восстанавливает полный граф объектов (команды + пилоты). Метод `GetAllDrivers()` возвращает всех пилотов, отсортированных по очкам.
- **Delete** — метод `DeleteTeam(int id)` удаляет запись из `Teams`. Каскадное удаление (ON DELETE CASCADE) автоматически очищает связанные таблицы-деталей и пилотов.

Методы классов первой работы в GUI

Методы доменной логики, разработанные в Лаб. 1, активно используются в GUI:

- `CalculatePoints(position)` — вызывается в `RaceSimulationWindow` для начисления очков после симуляции финиша;
- `AddPoints()`, `AddWin()`, `AddPodium()` — обновляют состояние объекта и сохраняются в БД через `SaveTeam()`;
- `GetWinRate()` — отображается в окне сравнения команд;
- Перегруженный оператор `>` — используется при сортировке списка команд по позиции в чемпионате.

Вывод

В ходе выполнения лабораторной работы была разработана полноценная кроссплатформенная GUI-система на языке C# с использованием Avalonia UI. Иерархия классов из Лаб. 1 перенесена без изменений: базовый абстрактный класс `FormulaTeam` с тремя дочерними (`F1Team`, `F2Team`, `FormulaETeam`), каждый в отдельном файле,

доступ к полям только через свойства. Данные хранятся в нормализованной СУБД SQLite по стратегии Table-Per-Type. Реализованы CRUD-операции, каскадное удаление и начальное заполнение БД (seeding). Методы предметной области из первой работы используются непосредственно в интерактивных функциях GUI: симуляция гонки, сравнение команд и управление личным зачётом пилотов.

Приложение A1. Базовый класс FormulaTeam.cs

<MINTED>

Приложение A2. Класс F1Team.cs

<MINTED>

Приложение A3. Класс F2Team.cs

<MINTED>

Приложение A4. Класс FormulaETeam.cs

<MINTED>

Приложение A5. Класс FormulaDriver.cs

<MINTED>

Приложение A6. Класс DatabaseManager.cs

<MINTED>

Приложение A7. MainViewModel.cs

<MINTED>

Приложение A8. TeamEditorViewModel.cs

<MINTED>

Приложение A9. MainWindow.axaml.cs

<MINTED>